

Button Control

Button Control

1. Learning Objectives
2. Hardware Setup
 - Independent Button Principle
 - Button Debouncing
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. Configure Pin Mode
 3. Write Program
 4. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Learn the basic working principle of buttons on the MSPM0G3519 motherboard.
2. Use the onboard button to control the onboard LED light on/off.

2. Hardware Setup

Independent Button Principle

Independent buttons are typically implemented through **mechanical switches**. Their hardware structure consists of a button, connecting wires, and pull-up/pull-down resistors. When the button is closed, the circuit is connected and the signal level changes; when the button is released, the circuit is disconnected and the signal returns to its initial state.

Button Debouncing

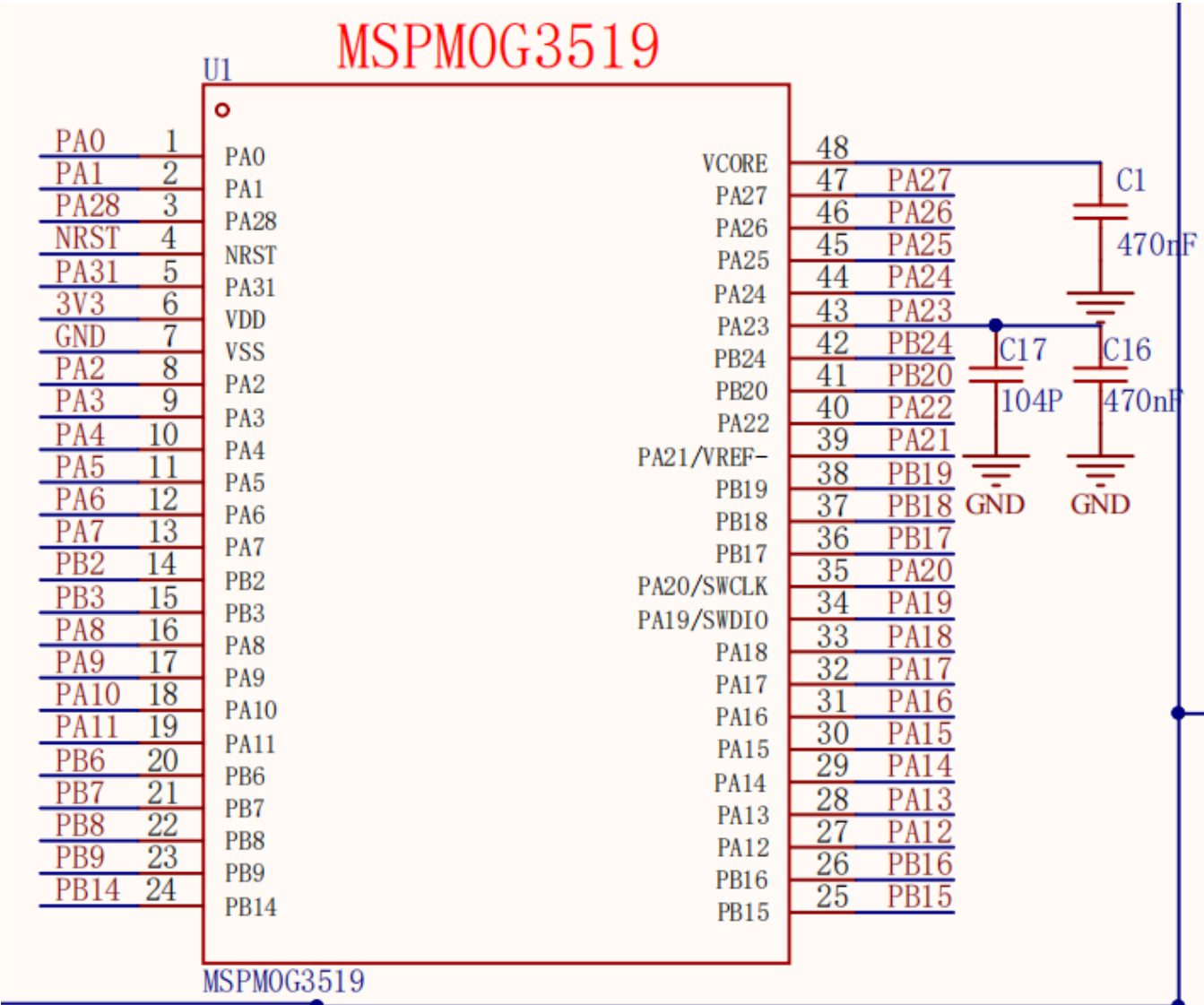
The mechanical elasticity of buttons causes the signal to jump multiple times in a short period when closing or releasing. This is called **button bouncing**. It is usually eliminated through **software delay debouncing** or hardware **RC filter circuit**.

This course does not require additional hardware. You can directly use the onboard function button on the MSPM0G3519 development board.

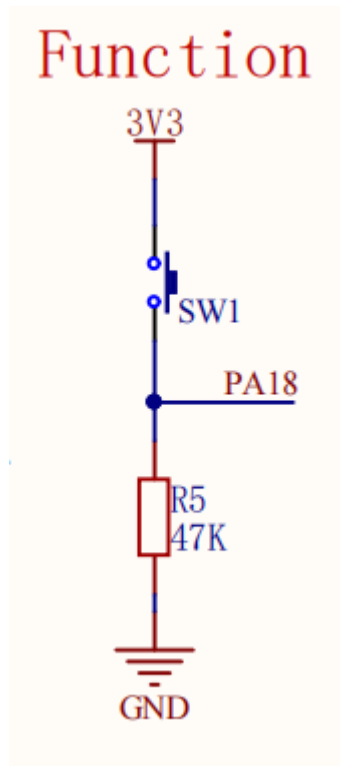
In the development board schematic, the PA18 pin is connected to a button. One end of the button is connected to high level (3.3V), and the other end is connected to the GPIO pin, while also being connected to ground through a 47k Ω pull-down resistor R5. When the button is not pressed, the PA18 pin level is pulled low to 0V due to the pull-down resistor. When the button is pressed, the PA18 pin is directly connected to high

level 3.3V, thereby achieving high and low level changes. The development board determines whether the button is pressed by detecting the level status of the PA18 pin. This design is simple and stable, effectively avoiding signal interference caused by floating pins.

MSPM0G3519 Main Control Diagram



Button Part Schematic

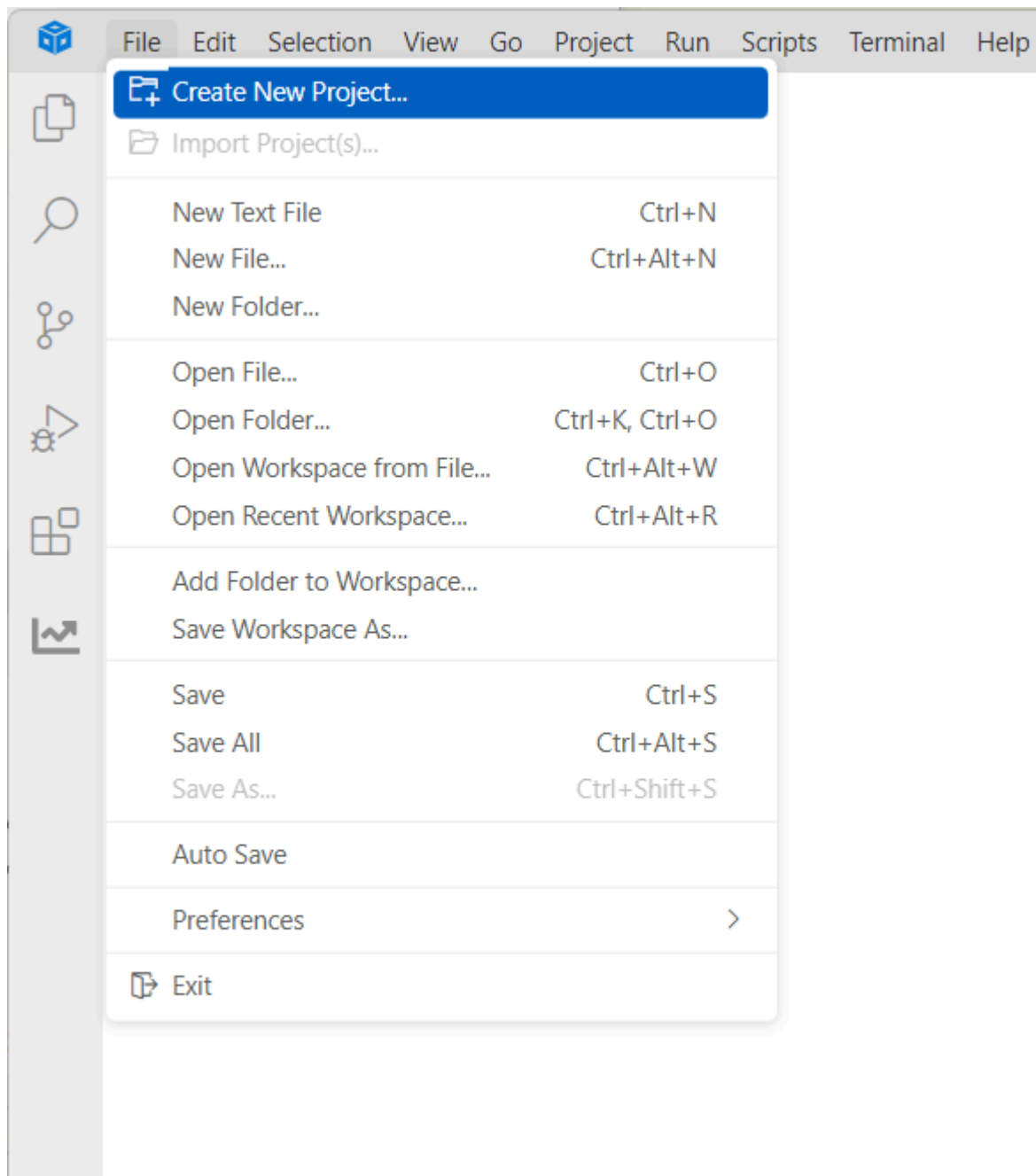


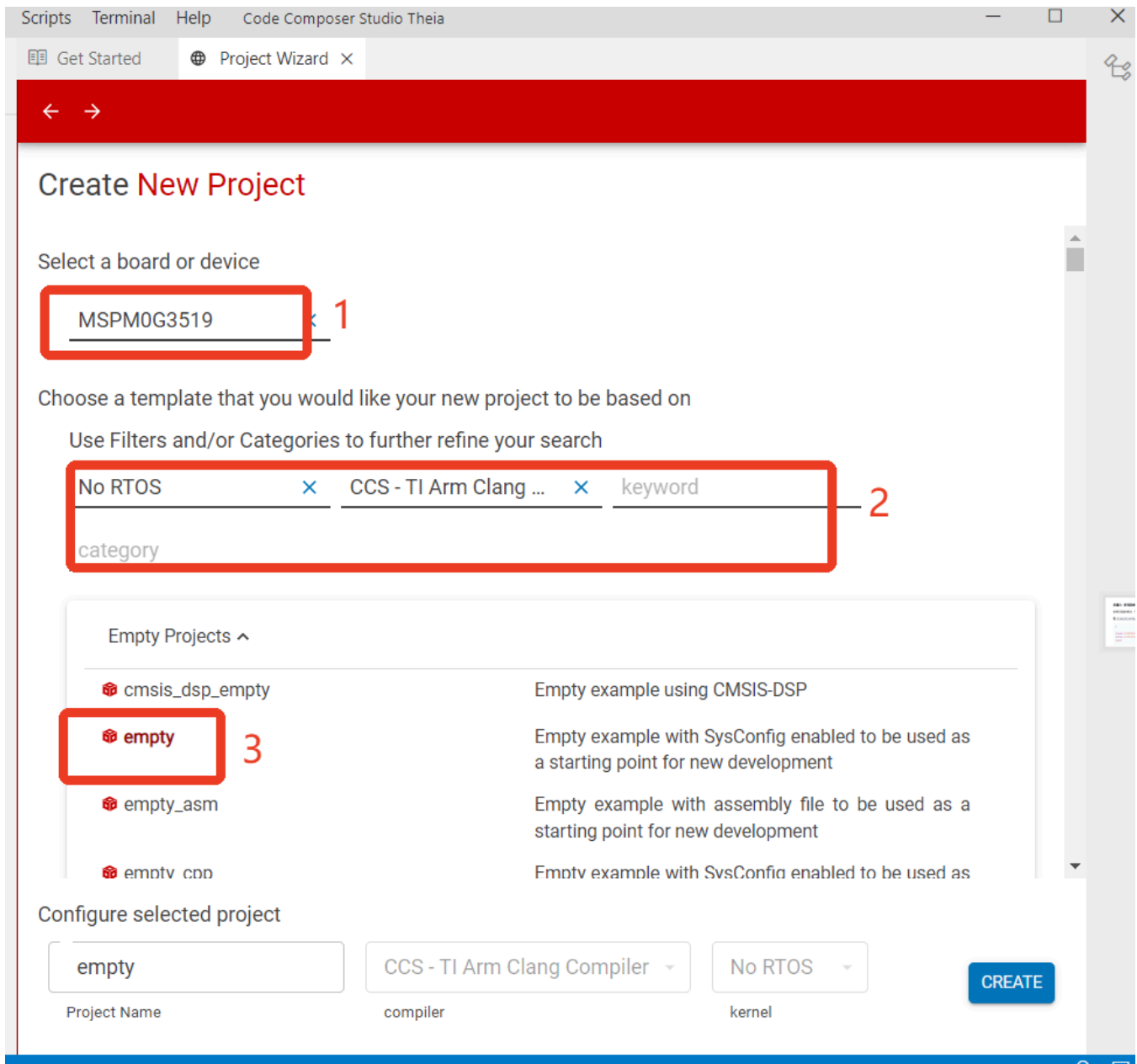
3. Experiment Steps

In microcontrollers, pin input mode means configuring a certain pin as an input function to receive changes in external signals. The working principle of buttons is based on this: by setting the pin connected to the button as input mode, the microcontroller can detect level changes on the pin. When the button is not pressed, the pin level remains low; when the button is pressed, the pin level becomes high. This high and low level change is the embodiment of the button input signal, and it is also the basis for the microcontroller to determine button status. Therefore, the process of receiving external signals is actually the process of detecting button high and low level changes through input mode.

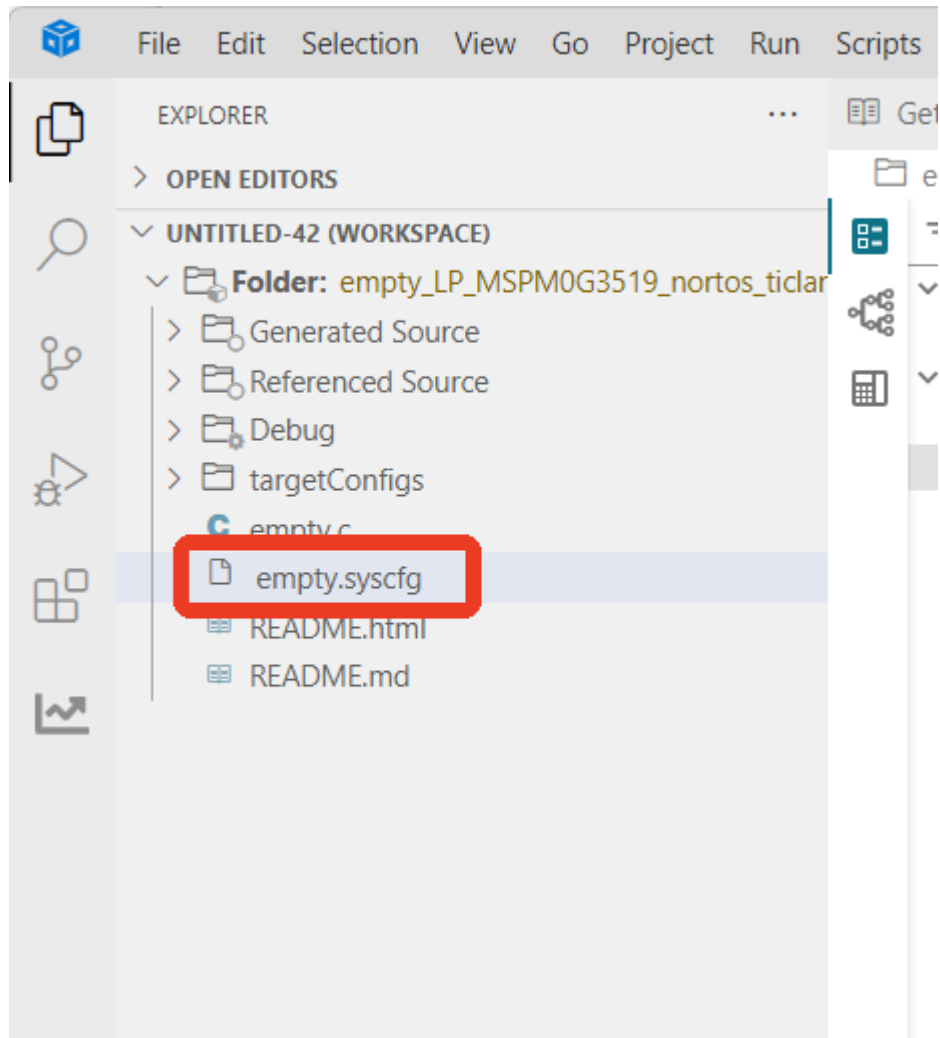
1. Open SYSCONFIG Configuration Tool

Create a new empty project empty in CCS.





Find and open the empty.syscfg file in the left workspace of CCS.



In SYSCONFIG, select MCU peripherals on the left, find the GPIO option and click to enter. Click ADD in GPIO to add two groups of pin peripherals: one for LED pins and one for button pins. For LED pin configuration, refer to the previous **Turn On LED** course. Only button pin configuration is explained here.

The screenshot displays the TI Sysconfig tool interface for an MSPM0G3519 device. The left sidebar shows the 'PROJECT CONFIGURATION' tree with 'GPIO' selected. The main workspace shows the 'GPIO (0 Added)' section, where a red box highlights the 'ADD' button. Below this, a diagram illustrates the GPIO module's internal components (DOUT, DOE, DIN, POLARITY, CTL, FASTWAKE, FILTEREN, DMAMASK) and its connections to the IOMUX, CPU, and Event modules. At the bottom, a form for adding a new GPIO instance is visible, with fields for Name (GPIO_GRP_0), Port (Any), Port Segment (Any), and Group Pins.

2. Configure Pin Mode

Configure pin parameters, name the GPIO instance `KEY`. Since it is connected to PA18, Port selects `PORTA`.

Set the pin name to `PIN_18`, set to input mode `Input`. Internal Resistor sets to pull-down resistor `Pu11-Down Resistor`. Assigned Pin sets the controlled pin to 18.

✓ KEY

Name

KEY

Port

PORTA

Port Segment

Any

Group Pins

1 added

+

 ADD

REMOVE ALL

✓ PIN_18

Name

PIN_18

Direction

Input

IO Structure [?]

Any

Digital IOMUX Features

Internal Resistor

Pull-Down Resistor

Invert

Disabled

Input Filter

Disabled

Enable Fast-Wake

☐

Override Pin Hysteresis Default

☐

Hysteresis Control

Disabled

Wakeup Logic

Disabled

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

18

Interrupts/Events

LaunchPad-Specific Pin

No Shortcut Used

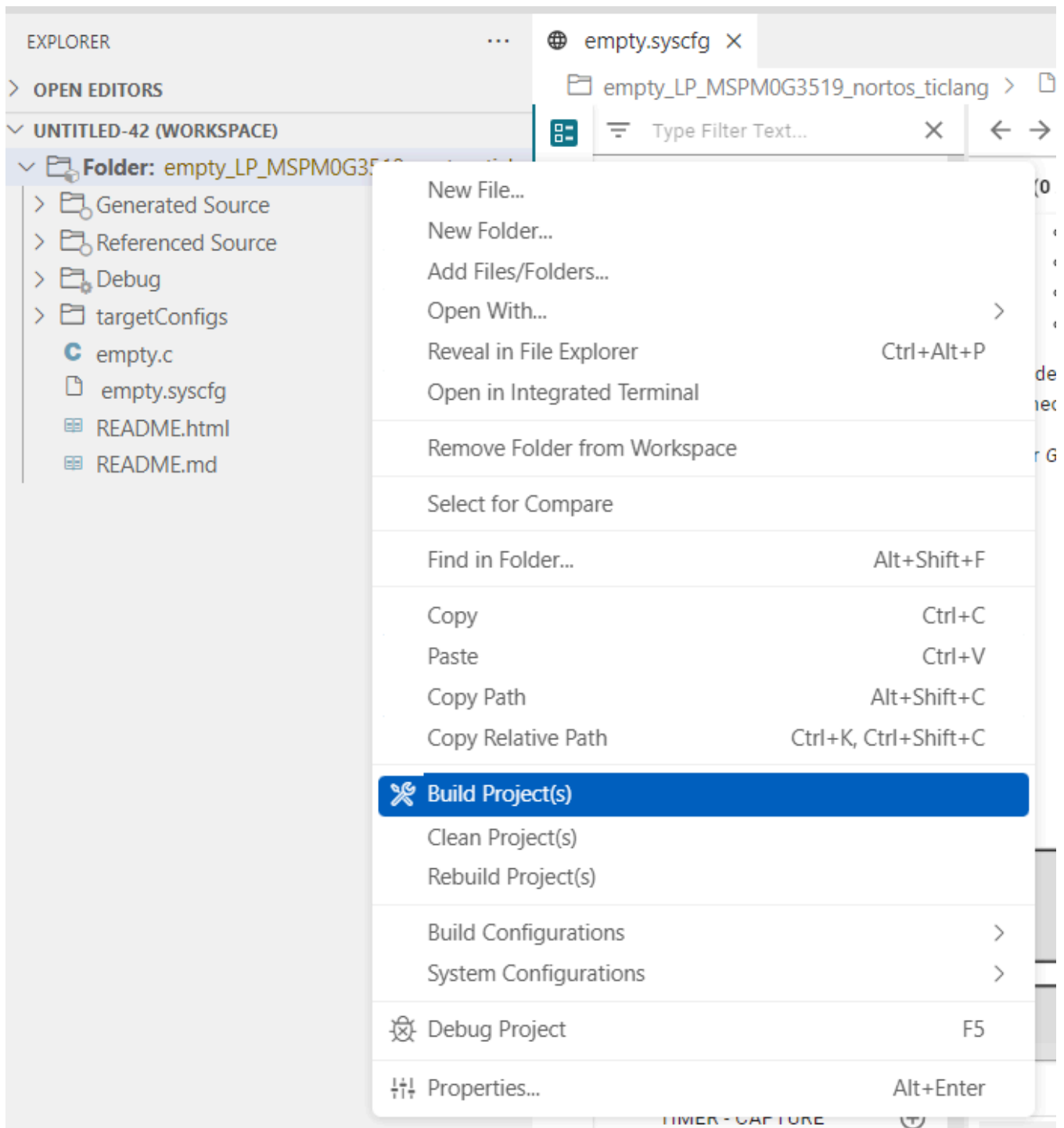
PinMux Peripheral and Pin Configuration

PIN_18

Any(PA18/11)

Save the configuration in the .syscfg file using the shortcut key Ctrl+S.

After saving, compile.



3. Write Program

In the empty.c file, write the following code

```
#include "ti_msp_dl_config.h"

int main(void)
{

    SYSCFG_DL_init();

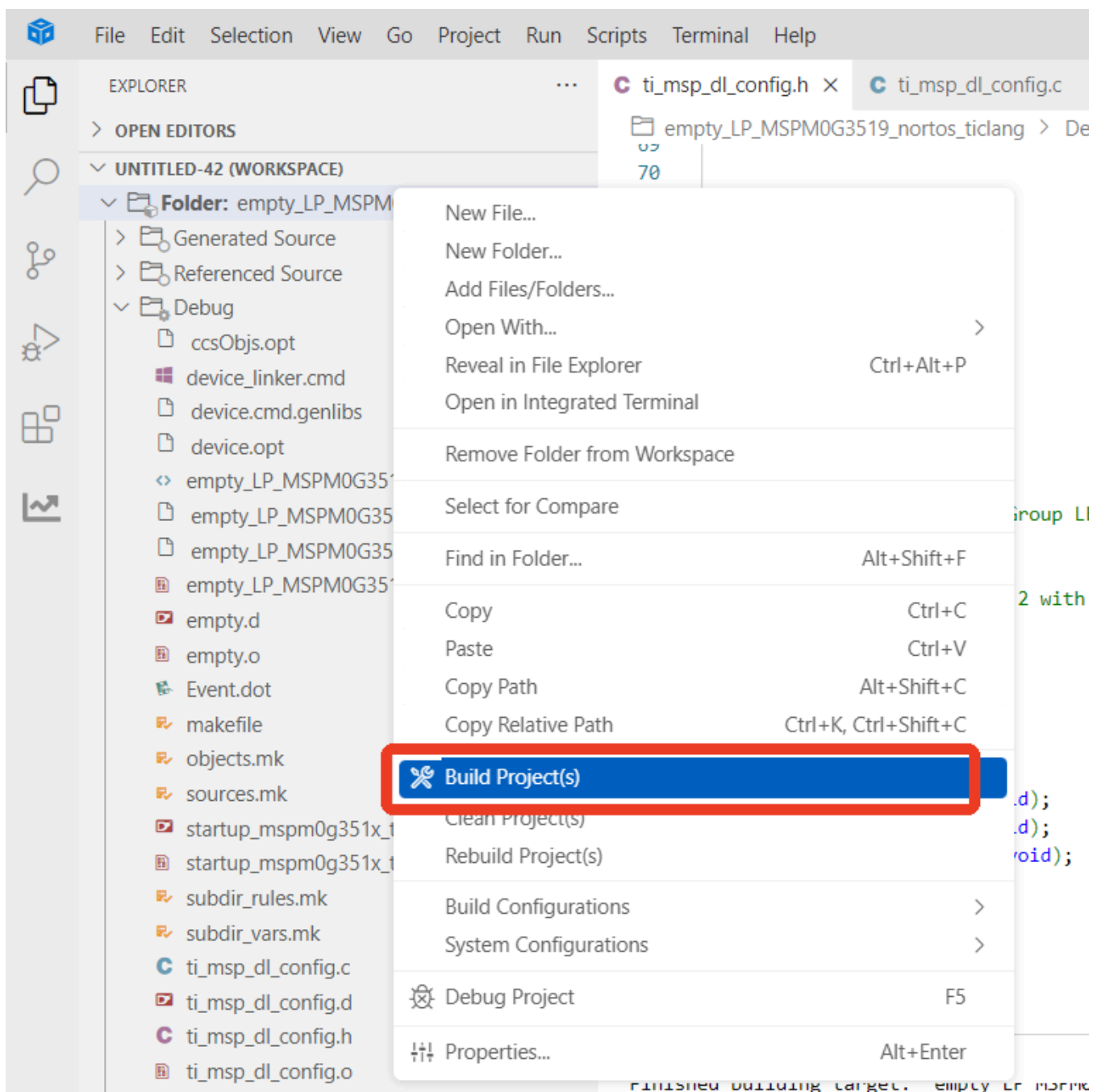
    while (1)
    {
```

```

        // If the pin value read is greater than 0, it means that the PA18 pin is
high level.
        if( DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN) > 0 )
        {
            DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //LED control output
high level
        }
        else// If the PA18 pin is low
        {
            DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //LED control output low
level
        }
    }
}

```

4. Compile



Compilation was successful. You can now download the program to the development board.

4. Program Analysis

- empty.c

```
03_key > C empty.c > main
1  #include "ti_msp_dl_config.h"
2
3  int main(void)
4  {
5
6      SYSCFG_DL_init();
7
8      while (1)
9      {
10         //如果读取到的引脚值大于0, 说明PA18引脚为高电平
11         // If the pin value read is greater than 0, it means that the PA18 pin is high level.
12         if( DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN) > 0 )
13         {
14             DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //LED控制输出高电平 LED control output high level
15         }
16         else//如果PA18引脚为低电平 If the PA18 pin is low
17         {
18             DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //LED控制输出低电平 LED control output low level
19         }
20     }
21 }
```

uint32_t DL_GPIO_readPins(GPIO_Regs* gpio, uint32_t pins) is used to read the level status of specified GPIO pins. By reading the value of the GPIO register, it obtains the status of the specified pin. Input the GPIO port and pin number to read, and return the level status of that pin.

gpio: The GPIO port to read.

pins: The pin number to read.

This code controls the LED by continuously detecting the level status of pin PA18 (button input pin) to control the output level of LED1_PIN_2 (LED control pin). When PA18 is high, the LED turns on (LED pin outputs high level); when PA18 is low, the LED turns off (LED pin outputs low level).

5. Experimental Phenomena

After the program is downloaded, you can control the LED light on/off with the button. Press the button, the LED turns on; release the button, the LED turns off.

